
Simple, Fast, and Practical Concurrent Queue Algorithms

Based on Michael and Scott, 1996

Hugo Wan

Motivation: Why Concurrent Queues Matter

- FIFO queues are widely used in parallel applications and operating systems.
- Multiple processes may enqueue and dequeue at the same time.
- Concurrent access must be synchronized to avoid corrupted queue state.
- A delayed process can hurt performance if the queue relies on blocking synchronization.
- The paper studies practical queue designs that are simple, fast, and safe.



Multiple processes can access the same FIFO queue concurrently, so the queue needs synchronization to preserve correctness.

Blocking vs. Non-Blocking Queues

Queue Design	Main Mechanism	Strength	Limitation
Single-lock queue	One global lock	Very simple	Only one operation can proceed at a time
Two-lock queue	Separate head and tail locks	One enqueue and one dequeue can overlap	Still vulnerable to delayed lock holders
Non-blocking queue	CAS / LL-SC atomic operations	System can keep making progress despite delays	More complex pointer logic
Earlier non-blocking queues	Snapshots or reference counting	Avoid ordinary locks	Higher overhead or memory-management issues

The paper compares blocking and non-blocking queue styles to show why a practical lock-free queue is useful under contention and process delays.

CAS and the ABA Problem

- The non-blocking queue relies on **compare-and-swap**, or **CAS**.
- CAS changes a shared value only if it still matches the expected old value.
- This allows pointer updates without locks.
- CAS-based algorithms must handle the **ABA problem**.
- The paper stores pointers with modification counters to reduce ABA risk.

1 Compare-and-Swap (CAS)

```
CAS(address, expected, new)
```

```
if address == expected:  
    address = new  
    success  
else:  
    fail
```

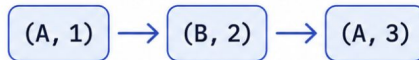


2 ABA problem



The pointer looks unchanged, but the structure changed in between.

3 Counter solution

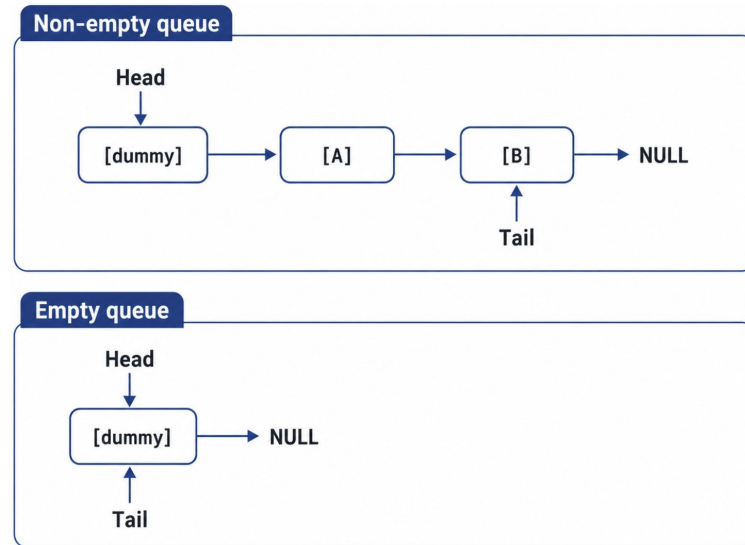


Add a modification counter to distinguish repeated pointer values.

CAS enables lock-free pointer updates, but pointer-count pairs are used to reduce the chance that an ABA change is mistaken as no change.

Non-Blocking Queue Data Structure

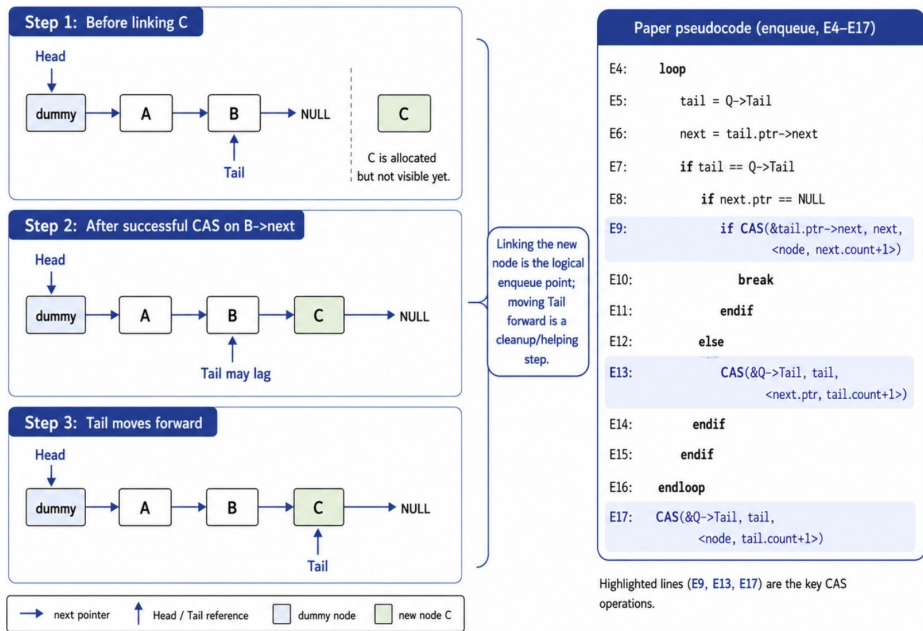
- The queue is implemented as a singly linked list.
- **Head** and **Tail** are shared pointer-count pairs.
- **Head** always points to a dummy node.
- The first real queue item is stored at **Head->next**.
- The dummy node simplifies empty and single-item queue cases.



The non-blocking queue uses a dummy node at the head of the linked list, so dequeue can advance **Head** instead of removing a special first node.

Enqueue Operation: Link First, Then Update Tail

Enqueue in the Michael–Scott Non-Blocking Queue

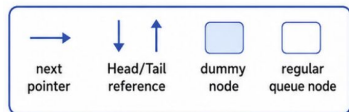
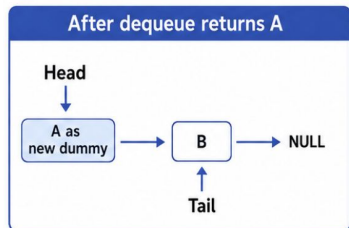
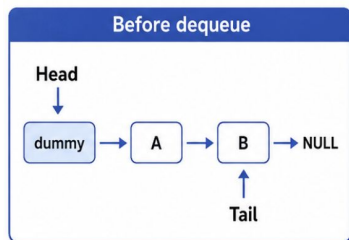


- Enqueue allocates a new node and sets its **next** pointer to **NULL**.
- It reads the current **Tail** and **Tail->next**.
- If **Tail->next == NULL**, the process tries to link the new node using CAS.
- The successful link is the logical enqueue point.
- Updating **Tail** afterward is a cleanup step, not the moment the item enters the queue.

Enqueue takes effect when CAS links the new node at the end of the list; moving **Tail** forward only helps future operations find the end faster.

Deque Operation: Move Head Forward

Deque in the Michael–Scott Non-Blocking Queue



Deque reads the value from the node after Head, then swings Head forward. The old dummy node can be freed after CAS succeeds.

Paper pseudocode (dequeue, D6–D13)

```
D6: if head.ptr == tail.ptr
D7:   if next.ptr == NULL
D8:     return FALSE
D9:   endif
D10:  CAS(&Q->Tail, tail,
        <next.ptr, tail.count+1>)
D11: else
D12:   *pvalue = next.ptr->value
D13:   if CAS(&Q->Head, head,
            <next.ptr, head.count+1>)
```



Key lines: D6–D8 check empty queue, D10 helps Tail catch up, D12–D13 perform the real dequeue.

- Dequeue reads **Head**, **Tail**, and **Head->next**.
- If **Head == Tail** and **Head->next == NULL**, the queue is empty.
- Otherwise, the value is read from **Head->next**.
- The successful CAS moving **Head** forward is the logical dequeue point.
- Afterward, the old dummy node can be freed.

Deque returns the value in **Head->next**, then advances **Head** so the removed node becomes the new dummy node.

Helping Mechanism and Non-Blocking Progress

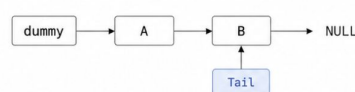
- **Tail** is allowed to temporarily lag behind the real last node.
- If another process sees this, it helps advance **Tail**.
- This appears in both enqueue and dequeue paths.
- A failed CAS usually means another process changed the structure first.
- This is why the system as a whole continues making progress.

Helping Mechanism

When **Tail** falls behind, other processes help advance it.

1. Tail falls behind

Head -> dummy -> A -> B -> NULL



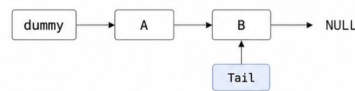
2. Another process helps

Performs CAS to move **Tail** forward.

CAS(Q->Tail, Tail, Tail->next)

3. After helping

Head -> dummy -> A -> B -> NULL



Key idea: Instead of waiting for the original enqueue/dequeue, any process can help advance **Tail** to keep the system moving.

Tiny crops from paper Figure 1:

E13: advancing Tail during enqueue

E13: CAS(&Q->Tail, tail, <next.ptr, tail.count+1>)

If **Tail** is not at the last node, a process helps advance **Tail**.

D10: advancing Tail during dequeue

D10: CAS(&Q->Tail, tail, <next.ptr, tail.count+1>)

If **Head** == **Tail** but **next** != NULL, **Tail** is falling behind, so a process helps advance **Tail**.

Why this helps:

Helping ensures that some operation completes in finite time (non-blocking progress).

When **Tail** lags behind, other processes can help advance it instead of waiting for the delayed process that created the lag.

Results: Dedicated vs. Multiprogrammed Systems

The performance graphs show that the new non-blocking queue remains the strongest overall result, while blocking algorithms suffer more as multiprogramming increases.

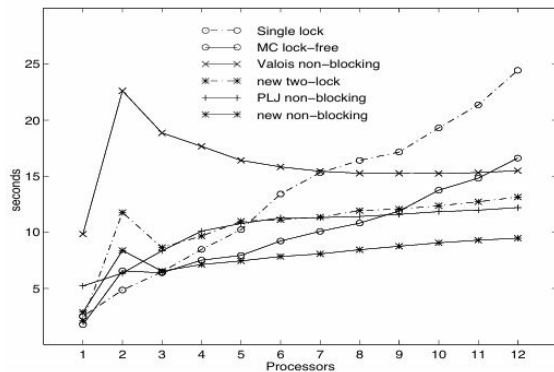


Figure 3: Net execution time for one million enqueue/dequeue pairs on a dedicated multiprocessor.

Figure 3:
Dedicated multiprocessor

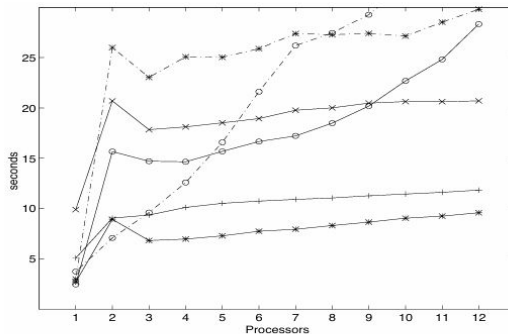


Figure 4: Net execution time for one million enqueue/dequeue pairs on a multiprogrammed system with 2 processes per processor.

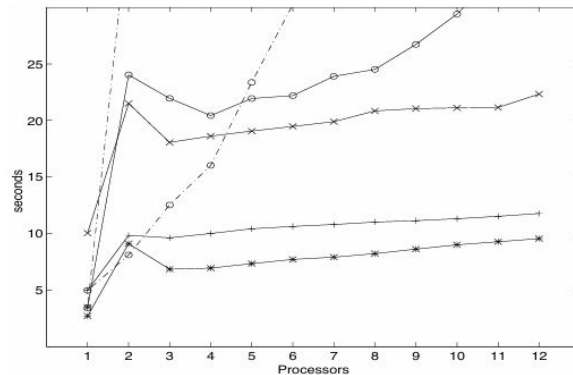


Figure 5: Net execution time for one million enqueue/dequeue pairs on a multiprogrammed system with 3 processes per processor.

Figure 5:
Multiprogrammed system with 3
processes per processor

Results: Dedicated vs. Multiprogrammed Systems

Paper Figure	Setup	Main Observation
Figure 3	Dedicated processors	New non-blocking queue performs best overall after several processors
Figure 4	2 processes per processor	Blocking algorithms degrade more under multiprogramming
Figure 5	3 processes per processor	Non-blocking advantage becomes strongest

The main trend across Figures 3–5 is that process delays make blocking queues less reliable, while the non-blocking queue remains more stable.

Thank You For Listening!!

- **Reference**

Maged M. Michael and Michael L. Scott.

“Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms.”

Department of Computer Science, University of Rochester.